Write Scala programs to solve the following problems. Do not use Spark. Try to use appropriate data structures and avoid using traditional Java-style loops if possible.

NOTE: I tried to use all immutable Maps and TreeMaps. My imports for the following code is as follows:

```
import scala.io.Source
import scala.collection.immutable.Map
import scala.collection.immutable.TreeMap
import scala.collection.immutable.TreeSet
```

Q1[10]. The program reads a file full of integers and computes the number of integers that are divisible by 3. Example input:

23 234 234 2 23
234 234 22 2 32324
234 23432

```
    val inputPath: String= "inputs/q1"

    // Convert each line to an element in a list
    val ints: List[String] = Source.fromFile(inputPath).getLines.toList

    // Each line is flattened into a 1D array of integers
    // Then that list is counted with a filtering condition (divisible by
3)
    val count: Int = ints.flatMap(_.split(" ").toList).count(_.toInt%3 ==
0)

    // Print the result
    println(count)
```

Q2[10]. The program reads a file that contains the date and a temperature that is measured for this day. There can be multiple temperatures that are measured each day. The program should print to the screen the highest temperature for each day. Example input:

2022/01/03 20
2022/01/03 30
2022/01/05 23

```
    val inputPath: String= "inputs/q2"

    // 2D array (the second array [0] is the date, and [1] is the
temperature)
```

```
    val dateTemps =
Source.fromFile(inputPath).getLines.toList.map(_.split(" ").toList)

    // Map the temperatures to the dates
    var dates = Map[String, String]();
    for (dateTemp <- dateTemps) {
      val d: String = dateTemp(0)
      // If the key is already in the map
      if (dates.contains(d)) {
        if (dateTemp(1).toInt > dates(d).toInt) {
          dates = dates + (d -> dateTemp(1))
        }
      }
      else {
        dates = dates + (d -> dateTemp(1))
      }
    }

    // Print all of the key-value pairs
    dates.foreach(println)
```

Q3[10]. Consider an input file with the following example input.

```
John Back, 23, B, CSC366
Bob Wilson, 11, B, CS201
John Back, 23, A, CSC369
```

In general, the input file will contain the student name, the student ID number, grade, and course. The program should print the student name, student id, and the list of classes for that student. The output should be **sorted by** name and then sorted by grade for each name. Here is example output.

```
Bob Wilson, 11, (B, CS201)
John Back, 23, (A, CSC369), (B, CSC366)
                            // sorted by name and then by grade5
```

```
    val inputPath: String= "inputs/q3"

    // 2D array
    // Nested array second array
    //  [0]: name
    //  [1]: id
    //  [2]: grade
```

```
    //  [3]: course
    // Each line has the previous information
    val lines = Source.fromFile(inputPath).getLines.toList.map(_.split(",
").toList)

    // Map the (grade, course) tuples to a given student (name, id)
    // Sorted on the key first
    var students = TreeMap[(String, Int), TreeSet[(String, String)]]();

    for (line <- lines) {
      // Student tuples will be (name, id) pairs
      val student: (String, Int) = (line(0), line(1).toInt)
      // Course tuples will be (grade, course) pairs
      val course: (String, String) = (line(2), line(3))

      // If the key is already in the map
      if (students.contains(student)) {
        students = students + (student -> (students(student) + course))
      }
      else { // Otherwise, create a new TreeSet
        students = students + (student -> TreeSet(course))
      }
    }

    // Print every student and the courses that they have taken
    students.foreach({
      case (student, courses) =>
        println(s"${student._1}, ${student._2}, ${courses.mkString(", ")}")
    })
```